

WIE

Native JavaScript Application

Microservice Architecture Reference

Complete Architecture Document

9 Microservices | Native JS Client | WebSocket + REST

Services / Hooks / Pages / Components / Types / Utils

Table of Contents

Table of Contents	2
1. Architecture Overview	4
1.1 Communication Patterns	4
1.2 Tech Stack Summary	4
2. Project Folder Structure	5
2.1 Root Layout	5
2.2 Services Layer	5
2.3 Hooks Layer	5
2.4 Pages Layer	6
2.5 Components Layer	7
2.6 Types and Utils	8
3. Microservice Domains	10
4. API Endpoint Reference	12
4.1 Auth Service (/api/auth)	12
4.2 Media Service — Flux (/api/flux)	12
4.3 Media Service — Diary (/api/diary)	12
4.4 Chat Service (/api/chat)	13
4.5 Notification Service (/api/notification)	13
4.6 Ticket Service (/api/ticket)	14
4.7 Transaction Service (/api/transaction)	14
4.8 Follow Service (/api/follow)	14
5. Core Client Modules	15
5.1 HTTP Core (services/core/http.js)	15
5.2 WebSocket Manager (services/core/ws.js)	15
5.3 Global Event Bus (utils/eventEmitter.js)	16
5.4 SPA Router (src/router.js)	16
6. Hooks Reference	18
6.1 Auth Hooks	18
6.2 Media Hooks	18
6.3 Chat Hooks	18
6.4 Utility Hooks	18
7. JSDoc Type Contracts	20
7.1 Auth Types (auth.types.js)	20
7.2 Flux Types (flux.types.js)	20
7.3 Chat Types (chat.types.js)	20
7.4 Transaction Types (transaction.types.js)	20
8. Quick Reference Summary	22

File Count by Layer	22
Naming Conventions	22

1. Architecture Overview

Wie is a social media platform built with a native JavaScript (Vanilla JS) client and a microservice backend. Each service domain is independently deployable and communicates via REST, WebSockets, or internal gRPC. The client app communicates exclusively through HTTP fetch wrappers and a singleton WebSocket manager.

1.1 Communication Patterns

Pattern	Usage
REST (fetch)	All CRUD operations via typed fetch() wrappers
WebSocket	Real-time chat, notifications, presence, flux events
gRPC	Internal server-to-server inter-service communication
EventEmitter	In-app pub/sub bus — decouples UI components
LocalStorage	Token persistence, offline cache, user prefs
FormData / Multipart	Media uploads (images, videos) to media service

1.2 Tech Stack Summary

Layer	Technology / Approach
Language	Vanilla JavaScript (ES2022+, no framework)
Module system	ES Modules (import / export) via Vite bundler
Routing	Hash-based SPA router (custom, no library)
State	Global EventEmitter store — no Redux/MobX
HTTP client	Typed fetch() wrapper with auto token-refresh
WebSocket	Singleton WieSocket class with exponential backoff
Upload	FormData multipart to wie-media-service / Cloudinary
Styling	CSS custom properties, BEM naming
Build	Vite 5 with code-splitting per page route

2. Project Folder Structure

2.1 Root Layout

```
wie-app/
├── src/
│   ├── index.js           # App entry point, DOM mount
│   ├── app.js             # Root component, layout shell
│   ├── router.js          # Hash-based SPA router
│   ├── store.js           # Global state via EventEmitter
│   ├── config/
│   │   ├── api.config.js  # Base URLs per microservice
│   │   ├── ws.config.js   # WebSocket endpoint map
│   │   └── env.js         # Environment variables
│   ├── services/
│   ├── hooks/
│   ├── pages/
│   ├── components/
│   ├── types/
│   ├── utils/
│   └── assets/
├── index.html             # HTML shell with #app mount
├── vite.config.js         # Build config, code splitting
└── package.json
```

2.2 Services Layer

```
src/services/
├── core/
│   ├── http.js           # fetch() wrapper + interceptors
│   ├── ws.js             # Singleton WebSocket manager
│   └── interceptors.js    # Auth header injection
├── authService.js        # wie-user-service: login, register, OTP
├── userService.js        # Profile CRUD, account privacy
├── mediaService.js       # Flux, diary, posts, reels
├── chatService.js        # Messages, rooms, group chats
├── notificationService.js # Push, in-app, badge counts
├── ticketService.js      # Support tickets, replies
├── transactionService.js # Wallet, topup, transfer, refund
├── followService.js      # Follow, unfollow, close friends
├── searchService.js      # User search, content discovery
├── eventService.js       # Events, RSVP, categories
└── uploadService.js      # Multipart FormData uploader
```

2.3 Hooks Layer

```
src/hooks/
├── auth/
│   ├── useAuth.js        # user, isLoading, isAuthenticated
│   ├── useLogin.js       # login() → token store + redirect
│   └── useRegister.js     # multi-step registration flow
```

```

├── useSession.js           # localStorage token, expiry guard
├── useOTP.js               # sendOTP(), verifyOTP() state machine
├── user/
│   ├── useProfile.js       # user, stats, fluxes, diaries
│   ├── useEditProfile.js   # updateProfile(), updateAvatar()
│   └── useFollowStats.js   # followers/following with optimistic updates
├── media/
│   ├── useFluxFeed.js      # feedGroups, refresh, CF-filtered
│   ├── useMyFluxes.js      # createFlux(), deleteFlux()
│   ├── useFluxActions.js   # like, archive, toggleComments
│   ├── useDiary.js         # diary CRUD, addFlux, reorder, pin
│   ├── useUpload.js        # upload(file) → progress%, url
│   └── useFluxViewer.js    # idx, progress, paused, tap zones
├── chat/
│   ├── useChat.js          # messages, send(), markRead(), WS
│   ├── useChatList.js      # chats, unreadCount, createChat()
│   └── useChatSocket.js    # WS lifecycle, reconnect, events
├── notifications/
│   └── useNotifications.js  # list, unreadCount, markRead()
├── follow/
│   ├── useFollow.js        # followStatus, toggle(), pending
│   └── useCloseFriends.js  # list, add(), remove(), save()
├── tickets/
│   └── useTickets.js        # list, createTicket(), reply()
├── transactions/
│   └── useWallet.js         # balance, topup(), transfer()
├── search/
│   └── useSearch.js         # query, results, 300ms debounce
├── useInfiniteScroll.js     # IntersectionObserver page loader
├── useDebounce.js          # Debounced value for search inputs
├── useLocalStorage.js      # [value, set, remove] with JSON
├── useEventEmitter.js      # Global pub/sub auto-cleanup
├── useMediaQuery.js        # Responsive breakpoint boolean
└── useToast.js             # showToast(), dismiss(), queue

```

2.4 Pages Layer

```

src/pages/
├── auth/
│   ├── Login.js           # useLogin, form → /home on success
│   ├── Register.js        # 3-step: creds → username → OTP
│   ├── ForgotPassword.js  # useOTP → reset flow
│   └── OTPVerify.js       # 6-digit input, resend timer
├── home/
│   └── Home.js            # Flux bubbles row + infinite posts feed
├── profile/
│   ├── Profile.js         # Own profile, highlights, tabs
│   ├── UserProfile.js     # Other user, follow gate, private
│   └── EditProfile.js     # Avatar crop, bio, username, links
├── flux/
│   ├── FluxCreate.js      # Media picker, canvas, music, audience
│   ├── FluxView.js        # Progress bars, overlays, viewer panel
│   └── FluxArchive.js     # Expired vs active (persistent) grid
├── diary/
│   ├── DiaryNew.js        # Select fluxes → title + cover editor
│   └── DiaryView.js       # Own diary viewer with controls

```

```

├── DiaryOtherView.js      # Other user diary (read-only)
├── chat/
│   ├── ChatList.js        # useChatList, unread badges
│   ├── ChatRoom.js        # useChat + WS, infinite messages
│   └── ChatMedia.js       # Media gallery for a chat room
├── notifications/
│   └── Notifications.js    # Grouped by date, mark all read
├── search/
│   ├── Search.js          # Debounced user + content results
│   └── Explore.js         # Trending, suggestions, reel grid
├── settings/
│   ├── Settings.js        # Sidebar nav shell
│   ├── AccountSettings.js # Profile, security
│   ├── PrivacySettings.js # Account privacy toggle
│   ├── BlockedAccounts.js # Block list
│   ├── NotifSettings.js   # Push / email prefs
│   ├── FluxSettings.js    # Close friends list + suggestions
│   └── AppearanceSettings.js # Theme toggle
├── tickets/
│   ├── Tickets.js         # List, status filter, create button
│   └── TicketDetail.js    # Thread view, attach files, reply
├── transactions/
│   ├── Wallet.js          # Balance, topup/withdraw/transfer
│   └── TransactionHistory.js # Filter, date range, CSV export

```

2.5 Components Layer

```

src/components/
├── common/
│   ├── Button.js          # label, onClick, variant
│   ├── Input.js           # type, value, onChange, error
│   ├── Modal.js           # isOpen, onClose, children
│   ├── Toast.js           # message, type, duration
│   ├── Spinner.js         # size, color
│   ├── Avatar.js          # src, size, ring
│   ├── Badge.js           # text, color
│   ├── BottomSheet.js     # isOpen, onClose
│   ├── ConfirmDialog.js   # title, onConfirm
│   └── InfiniteList.js    # items, fetchMore
├── layout/
│   ├── Sidebar.js         # collapsed state
│   ├── Header.js          # title, actions
│   ├── BottomNav.js       # activeTab
│   ├── PageWrapper.js     # children, scroll container
│   └── SplitLayout.js      # left, right panes
├── flux/
│   ├── FluxCard.js        # flux, onLike
│   ├── FluxOverlays.js    # text / sticker / location layers
│   ├── FluxToolbar.js     # activeTool
│   ├── FluxMusicBar.js    # song, paused
│   ├── FluxViewer.js      # fluxes[], startIdx
│   ├── StoryProgress.js   # count, idx, pct
│   ├── LocationStickerBadge.js # label, x, y position
│   ├── TextLayerCanvas.js # layers[]
│   ├── MusicBadge.js      # title, artist
│   └── FilterPreview.js    # value

```

```

├── diary/
│   ├── DiaryCard.js           # diary, onClick
│   ├── DiaryViewer.js        # diary, idx
│   ├── DiaryHighlightRing.js # diaries[]
│   ├── DiaryAddFluxPicker.js # onSelect callback
│   └── FluxOverlayCanvas.js  # flux rendering
├── chat/
│   ├── ChatList.js           # chats[], onSelect
│   ├── ChatBubble.js         # message, isMine
│   ├── ChatInput.js          # onSend, onMedia
│   ├── TypingIndicator.js    # users[]
│   ├── ChatHeader.js         # participant
│   ├── MessageMedia.js       # url, type
│   └── ReadReceipt.js        # isRead
├── profile/
│   ├── ProfileCard.js        # user, stats
│   ├── ProfileAvatar.js      # src, fluxCount
│   ├── FollowButton.js       # status, onClick
│   ├── FollowModal.js        # userId, tab
│   ├── ProfileTabs.js        # userId
│   ├── HighlightsRow.js      # diaries[]
│   └── StatPill.js           # label, count
├── notifications/
│   ├── NotificationItem.js    # notif
│   ├── NotificationBadge.js   # count
│   ├── NotificationList.js    # items[]
│   └── NotifPrefsToggle.js    # type
├── tickets/
│   ├── TicketCard.js          # ticket
│   ├── TicketMessageBubble.js # msg
│   ├── TicketStatusBadge.js   # status
│   ├── TicketCreateForm.js    # onSubmit
│   └── AttachmentPreview.js   # files[]
└── transactions/
    ├── WalletCard.js          # balance
    ├── TxHistoryItem.js       # tx
    ├── TxStatusBadge.js       # status
    ├── TopupSheet.js          # onSubmit
    ├── TransferSheet.js       # onSubmit
    └── PaymentMethodCard.js   # method

```

2.6 Types and Utils

```

src/types/
├── auth.types.js           # User, AuthTokens, Session
├── user.types.js           # UserProfile, FollowStatus
├── flux.types.js           # Flux, TextLayer, Sticker
├── chat.types.js           # Chat, Message, MessageType
├── notification.types.js   # Notification, NotificationType
├── ticket.types.js         # Ticket, TicketMessage, TicketStatus
└── transaction.types.js    # Transaction, Wallet, TxType

src/utils/
├── formatters.js          # Dates, numbers, currency
├── validators.js          # Email, phone, username rules
└── storage.js             # localStorage abstraction

```

```
|— timeAgo.js           # Relative time (2h ago)
|— mediaHelpers.js      # Blob, Object URL, mime type
|— eventEmitter.js      # In-app pub/sub bus
```

3. Microservice Domains

Each microservice is an independent deployable unit. The native JS client speaks to each via a dedicated service module using the HTTP core wrapper.

#1
AUTH

wie-user-service

- Login, register, OAuth
- Token refresh, session management
- Account privacy (public / private)
- Profile management, avatar upload
- OTP verification, password reset

#2
MEDIA

wie-media-service

- Flux (stories) CRUD
- Diary / highlights management
- Post, reel, archive
- Cloudinary upload integration
- Visibility controls (CF, followers, public)

#3
CHAT

wie-chat-service

- 1-to-1 direct messaging
- Group chat rooms
- Media messages (image, video)
- Read receipts, typing indicators
- Real-time via WebSocket

#4
NOTIF

wie-notification-service

- Push notifications (FCM / APNs)
- In-app alert feed
- Notification preferences
- Badge / unread counts
- Per-type toggles

#5
TICKET

wie-ticket-service

- Support ticket creation
- Issue tracking and replies
- Status lifecycle (open → resolved)
- Agent assignment
- File attachments

#6
TXN

wie-transaction-service

- Payments and wallet management
- Transaction history
- Refunds and disputes
- Balance management
- Multiple payment methods

#7
FOLLOW

wie-follow-service

- Follow / unfollow actions
- Follower and following lists
- Follow requests (private accounts)
- Mutual connection detection
- Close friends list management

#8
SEARCH

wie-search-service

- User search by username / name
- Content discovery
- Trending hashtags
- Suggested users
- Debounced 300ms client queries

#9
EVENT

wie-event-service

- Create and manage events
- RSVP management
- Event categories
- Location-based discovery
- Event feed integration

4. API Endpoint Reference

4.1 Auth Service (/api/auth)

Method	Endpoint	Description
POST	/register	Create account, return tokens
POST	/login	Email/phone + password authentication
POST	/refresh-token	Rotate access token using refresh token
POST	/logout	Invalidate session server-side
POST	/forgot-password	Send OTP to email or phone
POST	/verify-otp	Validate submitted OTP code
POST	/reset-password	Set new password after OTP verified
GET	/me	Fetch current authenticated user profile
PATCH	/me	Update profile fields (bio, username, etc.)
PATCH	/me/privacy	Toggle account privacy (public / private)

4.2 Media Service — Flux (/api/flux)

Method	Endpoint	Description
POST	/flux/create	Upload flux with media file
GET	/flux/feed	Following users active fluxes feed
GET	/flux/mine	Own active fluxes list
GET	/flux/archive	Expired and persistent fluxes
GET	/flux/:id	Single flux detail
DELETE	/flux/:id	Soft delete flux
PATCH	/flux/:id/visibility	Set public / followers / close-friends / only-me
PATCH	/flux/:id/persistent	Toggle no-expiry pinned story
POST	/flux/:id/like	Toggle like on flux
POST	/flux/:id/archive	Toggle archive state
PATCH	/flux/:id/comments/toggle	Enable or disable commenting
GET	/flux/:id/viewers	View list and counts

4.3 Media Service — Diary (/api/diary)

Method	Endpoint	Description
POST	/diary/create	New diary with optional cover image

GET	/diary/user/:userId	User diaries (visibility-gated)
GET	/diary/:id	Single diary with flux entries
PATCH	/diary/:id	Edit title, cover, visibility
DELETE	/diary/:id	Soft delete diary
POST	/diary/:id/add-flux	Append flux to diary
DELETE	/diary/:id/flux/:fluxId	Remove flux from diary
PATCH	/diary/:id/reorder	Reorder flux array
PATCH	/diary/:id/pin	Toggle pin on profile
POST	/diary/highlight	Quick add-to-diary from flux view

4.4 Chat Service (/api/chat)

Method	Endpoint	Description
POST	/chat/create	Create or fetch existing 1-to-1 chat
GET	/chat/list	All chats for current user
GET	/chat/:id	Single chat metadata
GET	/chat/:id/messages	Paginated message history
POST	/chat/:id/send	Send text or media message
PATCH	/chat/:id/read	Mark messages as read
DELETE	/chat/:id/message/:msgId	Delete a message
POST	/chat/group/create	Create new group chat room
PATCH	/chat/group/:id/members	Add or remove group members

4.5 Notification Service (/api/notification)

Method	Endpoint	Description
GET	/notification/list	All notifications, paginated
GET	/notification/unread-count	Badge count number
PATCH	/notification/:id/read	Mark single notification as read
PATCH	/notification/read-all	Mark all notifications as read
DELETE	/notification/:id	Delete notification
PATCH	/notification/preferences	Update push and email toggles
POST	/notification/register-token	Register FCM or APNs device token

4.6 Ticket Service (/api/ticket)

Method	Endpoint	Description
POST	/ticket/create	Open new support ticket
GET	/ticket/list	User ticket list
GET	/ticket/:id	Ticket detail with message thread
POST	/ticket/:id/message	Add reply to ticket thread
PATCH	/ticket/:id/status	Update status: open/in-progress/closed
POST	/ticket/:id/attachment	Upload file attachment to ticket
GET	/ticket/categories	Category list for creation form

4.7 Transaction Service (/api/transaction)

Method	Endpoint	Description
GET	/transaction/wallet	Balance and wallet info
GET	/transaction/history	Paginated transaction list
GET	/transaction/:id	Single transaction detail
POST	/transaction/topup	Add funds to wallet
POST	/transaction/withdraw	Withdraw funds
POST	/transaction/transfer	Transfer to another user
POST	/transaction/:id/refund	Request refund on transaction
GET	/transaction/methods	Saved payment methods list

4.8 Follow Service (/api/follow)

Method	Endpoint	Description
POST	/follow/:userId	Follow user or send follow request
DELETE	/follow/:userId	Unfollow user
GET	/follow/:userId/followers	Follower list
GET	/follow/:userId/following	Following list
GET	/follow/requests	Pending follow requests
POST	/follow/request/:id/accept	Accept follow request
DELETE	/follow/request/:id	Decline or cancel request
GET	/follow/close-friends	Close friends list
POST	/follow/close-friends/save	Bulk save close friends list

5. Core Client Modules

5.1 HTTP Core (services/core/http.js)

A typed fetch() wrapper that injects Bearer tokens, auto-refreshes on 401, and exports method helpers.

```
// Auto-refreshing fetch wrapper
async function request(url, options) {
  const token = getAccessToken();
  const res = await fetch(BASE_URL + url, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`,
      ...options.headers,
    },
  });
  if (res.status === 401) {
    await refreshToken();
    return request(url, options); // retry once
  }
  if (!res.ok) throw await res.json();
  return res.json();
}

export const get    = (url, p) => request(url, { method: 'GET', ...p });
export const post   = (url, b) => request(url, { method: 'POST',  body:
JSON.stringify(b) });
export const patch  = (url, b) => request(url, { method: 'PATCH', body:
JSON.stringify(b) });
export const del    = (url)   => request(url, { method: 'DELETE' });
export const upload = (url, f) => fetch(BASE_URL + url, {
  method: 'POST',
  headers: { Authorization: `Bearer ${getAccessToken()}` },
  body: f, // FormData - no Content-Type override
});
```

5.2 WebSocket Manager (services/core/ws.js)

A singleton WieSocket class handles connection lifecycle, namespaced events, and exponential backoff reconnect.

```
class WieSocket {
  connect(token)      // Auth handshake on connect
  disconnect()        // Graceful close
  on(event, handler)  // Subscribe to event
  off(event, handler) // Unsubscribe from event
  emit(event, data)    // Send message to server
  reconnect()          // Exponential backoff retry
}

// Chat namespace events
```

```

ws.on('message:new',      handler) // New inbound message
ws.on('message:read',     handler) // Read receipt
ws.on('typing:start',     handler) // Typing indicator on
ws.on('typing:stop',      handler) // Typing indicator off
ws.on('chat:created',     handler) // New chat room created

// Notification namespace
ws.on('notification:new', handler) // New notification
ws.on('notification:count', handler) // Badge count update

// Flux namespace
ws.on('flux:view',        handler) // View registered
ws.on('flux:like',        handler) // Like toggle
ws.on('flux:comment',     handler) // New comment

// Presence
ws.on('user:online',      handler) // User came online
ws.on('user:offline',    handler) // User went offline

```

5.3 Global Event Bus (utils/eventEmitter.js)

In-app pub/sub decouples components without sharing state directly. Used for cross-domain side-effects.

```

emitter.emit('auth:logout')           // Trigger logout across app
emitter.emit('toast:show', { msg, type }) // Global toast notification
emitter.emit('flux:liked', { fluxId })  // Flux like state sync
emitter.emit('diary:updated', { diaryId }) // Diary content changed
emitter.emit('follow:changed', { userId }) // Follow state changed
emitter.emit('chat:unread', { count })  // Badge count update
emitter.emit('ticket:updated', { ticketId }) // Ticket status change
emitter.emit('tx:completed', { txId })  // Transaction done

// Usage in hooks:
useEventEmitter('auth:logout', () => {
  clearSession();
  router.push('/login');
});

```

5.4 SPA Router (src/router.js)

Hash-based SPA router with lazy-loading per route and a beforeEach auth guard.

```

const routes = {
  '/':          () => import('./pages/Home.js'),
  '/login':     () => import('./pages/auth/Login.js'),
  '/register':  () => import('./pages/auth/Register.js'),
  '/profile':   () => import('./pages/profile/Profile.js'),
  '/profile/:userId': () => import('./pages/profile/UserProfile.js'),
  '/flux/create': () => import('./pages/flux/FluxCreate.js'),

```

```
  '/flux/:id':      () => import('./pages/flux/FluxView.js'),
  '/diary/new':     () => import('./pages/diary/DiaryNew.js'),
  '/diary/:id':     () => import('./pages/diary/DiaryView.js'),
  '/chat':          () => import('./pages/chat/ChatList.js'),
  '/chat/:id':      () => import('./pages/chat/ChatRoom.js'),
  '/notifications': () => import('./pages/notifications/Notifications.js'),
  '/settings':      () => import('./pages/settings/Settings.js'),
  '/settings/privacy': () => import('./pages/settings/PrivacySettings.js'),
  '/settings/flux': () => import('./pages/settings/FluxSettings.js'),
  '/tickets':       () => import('./pages/tickets/Tickets.js'),
  '/tickets/:id':   () => import('./pages/tickets/TicketDetail.js'),
  '/wallet':        () => import('./pages/transactions/Wallet.js'),
};

// Auth guard - redirect to /login if no token
router.beforeEach((to) => {
  if (!getAccessToken() && to !== '/login') {
    router.push('/login');
  }
});
```

6. Hooks Reference

6.1 Auth Hooks

Hook	Returns / Args	Purpose
useAuth()	user, isLoading, isAuthenticated	Central auth state; auto-refreshes expired token
useLogin()	login(email, pass)	Stores tokens, emits auth:login, redirects to /home
useRegister()	register(data)	Multi-step: creates account, auto-logs in
useSession()	token, setToken, clear	Reads/writes token from localStorage, expiry guard
useOTP()	sendOTP(), verifyOTP(code)	OTP state machine: idle → sent → verifying → done

6.2 Media Hooks

Hook	Returns / Args	Purpose
useFluxFeed()	feedGroups, loading, refresh()	Following users fluxes, grouped; CF-filtered
useMyFluxes()	fluxes, createFlux(data), deleteFlux(id)	Own flux CRUD with optimistic state
useFluxActions(id)	like, archive, toggleComments, persist	All action toggles for a specific flux
useDiary(id)	diary, addFlux, removeFlux, reorder, pin	Full diary management including reordering
useUpload()	upload(file, type), progress, url	FormData multipart upload with progress%
useFluxViewer()	idx, progress, paused, goNext, goPrev	Viewer state machine for story playback

6.3 Chat Hooks

Hook	Returns / Args	Purpose
useChat(chatId)	messages, send(text), markRead()	Message list + WS-driven real-time updates
useChatList()	chats, unreadCount, createChat(userId)	Chat list with unread badge counts
useChatSocket()	connected, reconnect()	WS lifecycle manager, reconnect logic

6.4 Utility Hooks

Hook	Args	Purpose
useInfiniteScroll(fn)	fetchFn callback	IntersectionObserver-based page loader
useDebounce(val, ms)	value, delay ms	Returns debounced value (300ms for search)
useLocalStorage(key)	key string	[value, set, remove] with JSON serialisation
useEventEmitter(event)	event name, handler	Global pub/sub subscribe with auto-cleanup
useMediaQuery(query)	CSS media query string	Returns boolean for responsive breakpoints
useToast()	—	showToast(msg, type), dismiss(), stacked queue

7. JSDoc Type Contracts

All type contracts are defined as JSDoc `@typedef` blocks in `src/types/`. These are not runtime — they serve as in-editor IntelliSense contracts for VS Code.

7.1 Auth Types (`auth.types.js`)

Type	Key Fields	Notes
User	id, username, email, profile_picture, bio, accountPrivacy, posts_count	Core user identity object
AuthTokens	accessToken, refreshToken, expiresIn	Returned on login and refresh
Session	user: User, tokens: AuthTokens	Full session stored in memory + localStorage

7.2 Flux Types (`flux.types.js`)

Type	Key Fields	Notes
FluxVisibility	public followers close_friends only_me	Union string literal
FluxMediaType	image video text	Union string literal
FluxStatus	active expired archived deleted	Union string literal
Flux	_id, userId, mediaUrl, mediaType, visibility, commentsDisabled, isPersistent, textLayers, stickers, locationLabel, expiresAt, viewCount	Main flux object
TextLayer	id, text, x, y, color, font, fontSize	Canvas overlay text layer

7.3 Chat Types (`chat.types.js`)

Type	Key Fields	Notes
MessageType	text image video flux_ref	Union string literal
Message	_id, chatId, senderId, text, type, mediaUrl, fluxId, isRead, createdAt	Single chat message
Chat	_id, type, participant, lastMessage, unreadCount, updatedAt	Chat room descriptor

7.4 Transaction Types (`transaction.types.js`)

Type	Key Fields	Notes
------	------------	-------

TxType	topup withdraw transfer refund	Union string literal
TxStatus	pending success failed refunded	Union string literal
Transaction	_id, userId, type, status, amount, currency, toUserId, reference, createdAt	Transaction record
Wallet	userId, balance, currency, methods: PaymentMethod[]	User wallet with payment methods

8. Quick Reference Summary

File Count by Layer

Layer	Approx. File Count
src/services/	11 service files + 3 core
src/hooks/	24 hook files across 7 domains
src/pages/	28 page files across 12 sections
src/components/	44 component files across 8 groups
src/types/	7 JSDoc typedef files
src/utils/	6 utility files
src/config/	3 config files
Root	index.js, app.js, router.js, store.js

Naming Conventions

Item	Convention
Pages	PascalCase — Home.js, FluxView.js
Components	PascalCase — ChatBubble.js, WalletCard.js
Hooks	camelCase with use prefix — useAuth.js, useChat.js
Services	camelCase with Service suffix — authService.js
Types	PascalCase typedef, camelCase file — auth.types.js
Utils	camelCase — formatters.js, timeAgo.js
Events (WS)	namespace:action — message:new, flux:like
Events (bus)	domain:action — auth:logout, toast:show
CSS classes	BEM — .flux-card, .flux-card__overlay
Routes	kebab-case — /flux/create, /diary/new